

# Innovation Insight: AI-Augmented Code Modernization Tools

6 October 2025 - ID G00836092 - 15 min read

By Prasanna Lakshmi Narasimha

Modernizing legacy code is complex and time-consuming. However, software engineering leaders responsible for modernization initiatives can reduce time, effort, and risk compared to manual modernization using AI-augmented code modernization tools.

## Overview

### Key Findings

- Early previews of AI-augmented modernization tools demonstrate the ability to migrate legacy code to modern languages and frameworks, signaling potential for reduced manual work.
- AI-augmented code modernization tools vary significantly in capabilities, licensing model, vendor support, professional services, and total cost of ownership (TCO).
- While these tools utilize AI to automate and accelerate many tasks, human oversight remains crucial at various stages of the modernization life cycle.

### Recommendations

- Break the traditional iron triangle of cost, quality, and time of legacy code modernization efforts by helping your teams smartly wield AI-augmented code modernization tools in relevant use cases.
- Select the right tool for the job by evaluating which differentiating capabilities — such as legacy code documentation, business rules extraction, or code migration — best address the specific legacy situations present within your applications.
- Develop an implementation plan that includes a human in the loop (HiTL) to maintain control, ensure quality, validate complex logic, and retain accountability.

## Strategic Planning Assumptions

By 2029, organizations will complete 90% of software modernization using AI-augmented tools, a significant increase from less than 15% today.

By 2029, GenAI will reduce modernization labor costs by 50% compared with 2025 levels.

## Introduction

In an era where rapid delivery and scalable architectures define competitive advantage, outdated codebases and mounting technical debt threaten to slow innovation and inflate costs. 44% of enterprises identify the burden of technical debt as the second most common challenge among their top three concerns. Traditional modernization relies on manual code reviews, rule-based scripts and extensive developer effort — processes that are time-consuming, error-prone, and hard to scale.

AI-augmented code modernization tools empower software engineering leaders to tackle these challenges head-on. By harnessing machine learning and generative AI, these tools provide the following capabilities:

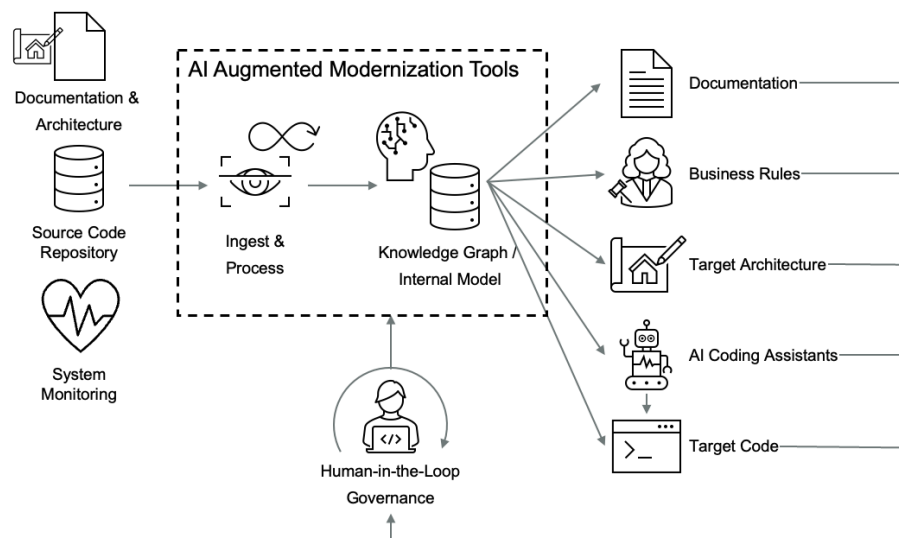
- Assess legacy codebases
- Modernization planning
- Build an understanding of the codebase and document it
- Extract business logic and business rules embedded in the legacy code
- Generate refactoring recommendations and code transformation
- Generate tests
- Streamline migration to modern languages and frameworks

While these capabilities can significantly accelerate modernization efforts, leaders must also carefully assess potential risks such as skill erosion, inaccurate transformations, and change management complexities to ensure successful outcomes (see Figure 1).

Figure 1: Application Modernization Using AI-augmented Modernization Tools



## Application Modernization Using AI-augmented Modernization Tools



Source:  
836092

**Gartner**

## Description

### AI-augmented application modernization tools

AI-augmented application modernization tools are technologies that leverage artificial intelligence (AI) to assist organizations in updating and improving their legacy applications. These tools aim to enhance the modernization process's efficiency, quality and speed by automating various tasks and providing intelligent insights.

AI-augmented application modernization tools have evolved and are more capable than rule-based or pattern-matching scripting approaches. Unlike rule-based or pattern-matching scripting approaches, GenAI-powered tools build a comprehensive internal model of the legacy application code. This deeper understanding enables them to offer enhanced capabilities across the discover, design and deliver phases as below:

#### • Discover

- **Discovery and assessment** — Automatically inventory legacy assets, map dependencies, score code complexity, security, and technical debt. For example, Mechanical Orchard Imogen monitors the application data flows to determine key dependencies, facilitating incremental rebuilds with changes that the tool verifies against actual runtime data.
- **Business rules extraction** — Extract business logic and business rules embedded in the legacy application code for AI-augmented code transformation or as a basis for manually rearchitecting or rewriting the application. For example, Evolveware Intellisys provides AI-driven extraction of embedded knowledge and business rules and automates documentation updates.
- **Documentation** — Document legacy applications to bootstrap comprehension of complex codebases by providing contextual insights, such as connections between different parts of code and implementation approach. This documentation can then serve as a resource for AI coding assistants, re-documenting the application, reverse engineering, or as a basis for manually rearchitecting or rewriting the application. For example, Phase Change provides precise information about the business functionality contained in the COBOL code to all stakeholders, or vFunction provides runtime architectural insights to power refactoring using GenAI code assistants.

#### • Design

- **Specifications** — Create product requirement documents that give non-technical stakeholders immediate access to the workings of the application and assist in rewriting code in the target language using the AI coding assistants. For example, CoreStory creates product requirements documents (PRDs) and uses them to generate target code.
- **Planning and design** — Use predictive analytics to recommend what to modernize first, estimate effort and forecast ROI.
- **Impact analysis** — Build an internal model of the application. For example, Devsu Metamorph ingests existing software assets and turns it into a graph, which can then be compared to an ideal target graph to identify changes' impacts or estimate the work.

#### • Delivery

- **Automated transformation** — Leverage Generative AI and code-analysis models to refactor or translate code, extract APIs, and containerize or rearchitect components.

- **Testing and validation** — Generate user stories, test cases, synthetic data, and regression suites, and detect anomalies or performance regressions. For example, IBM watsonx Code Assistant generates unit tests using a chat command.
- **Continuous monitoring and optimization** — Apply AIOps for runtime observability, performance tuning, cost-right-sizing and ongoing feedback to retrain models.
- **Built-in security and compliance** — Integrate AI-based SAST/DAST scanning, policy-as-code enforcement, and automated reporting throughout the modernization pipeline.

## Benefits and Uses

AI-augmented application modernization tools deliver structured advantages across four key legacy constraints — cost of change, optimized deployment and maintenance, agility and personalized experience, and intelligent portfolio investment — by automating code transformation, improving quality, and guiding strategic decision making.

- Cost of Change
  - **Accelerated Modernization:** Automated code analysis, refactoring, and translation speed up large-scale transformations and reduce time-to-value. For example, Moderne can tackle large codebases across thousands of projects to enable automated and scalable code transformations for framework migrations, security remediation, or swapping technology, e.g., message queues, databases, etc.
  - **Improved Code Quality and Consistency:** AI-driven code smell detection, dependency analysis, and style enforcement lower defect rates and eliminate rework by uniformly applying best practices.
- Optimized Deployment and Maintenance
  - **Faster, More Reliable Testing and Validation:** On-demand generation of test cases, synthetic data sets, and regression suites, plus automated anomaly detection, trims QA cycles and minimizes manual effort.
  - **Integrated Security and Compliance:** Continuous AI-driven vulnerability scanning (SAST/DAST), policy-as-code frameworks, and automated compliance reporting streamline ongoing maintenance and reduce risk.
- Agility and Personalized Experience
  - **Enhanced Developer Productivity:** Automating repetitive tasks frees engineers to innovate on business-critical features and respond rapidly to changing requirements.
  - **Preservation of Institutional Knowledge:** NLP-powered code annotations capture undocumented business rules and design intents, ensuring modernization doesn't erase critical tribal knowledge — and even enhances — the developer experience. For example, swimm.io provides a reliable and accurate view of the application based on static analysis and deterministic techniques.
- Intelligent Portfolio Investment
  - **Smarter Risk Assessment and Prioritization:** Predictive analytics score code modules by complexity, security exposure, and technical debt, guiding data-driven decisions on which components to modernize first.
  - **Scalable Coverage of Legacy Portfolios:** The ability to ingest and analyze millions of lines of code across dozens of languages, with automated dependency mapping, uncovers hidden service-and-data-flow risks and maximizes return on investment.

Table 1 lists the primary uses of AI-augmented application modernization tools, organized by phase.

| Primary uses of AI-augmented application modernization tools |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Application Modernization Phase                              | Primary Uses                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Discover                                                     | <ul style="list-style-type: none"><li>• Automated inventory of code, data stores, services, and external dependencies.</li><li>• AI-powered risk, complexity and technical-debt scoring across the portfolio to create cohesive modernization plans.</li><li>• NLP-driven extraction of undocumented business rules and workflows.</li><li>• Change-history tracking with AI-annotated rationales for transformations.</li><li>• Knowledge capture of legacy “tribal” rules to preserve institutional memory.</li><li>• Effective knowledge management (KM) in application modernization requires thorough documentation of existing assets — such as code, configurations, data models, and business processes — and systematic management of broader organizational knowledge, including architectural rationale, product design, marketing strategies, and policy frameworks. By intentionally designing KM processes to capture and utilize both technical and business insights, organizations enable precise modernization, foster innovation, and support strategic decision making.</li><li>• Detection of sensitive data flows and privacy violations.</li></ul> |
| Design                                                       | <ul style="list-style-type: none"><li>• Extracting business logic and business rules embedded in the legacy application code.</li><li>• Data-driven roadmaps that rank applications/modules by business value and refactoring effort</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

Source: Gartner (2025)

## Risks

Despite their transformative potential, AI-augmented application modernization tools present risks that software engineering leaders must proactively manage. Organizations group these risks into four categories to enhance clarity and enable actionable risk modeling: organizational, knowledge, technology, and process risks. Additionally, certain risks may be amplified or mitigated depending on specific use cases and modernization scenarios.

- **Organizational Risks**
  - **Overreliance and Skills Erosion:** GenAI-driven code modernization can cause developers to rely excessively on AI-generated refactoring and migration suggestions, diminishing their expertise. This risk intensifies in use cases involving highly customized legacy systems, where AI modernization tools may overlook nuanced challenges unless developers manually intervene.
  - **Vendor Lock-In and Interoperability:** Proprietary AI tooling can create dependency on a single vendor's APIs, data formats, and pipelines. This risk is particularly acute in large-scale modernization programs where switching costs and integration complexity are magnified.
- **Knowledge Risks**
  - **Loss of Context and Institutional Knowledge:** Models that primarily learn from code patterns may overlook domain-specific rules, regulatory requirements, or custom workflows. This risk increases in regulated industries and use cases involving mission-critical business logic, where excessive automation can undermine the rationale behind architectural decisions.
- **Technology Risks**
  - **Inaccurate or Hallucinated Transformations:** Misinterpreting legacy business logic can produce incorrect refactorings or API signatures. Silent failures — defects or misbehaviors that do not immediately trigger errors — are hazardous. For example:
    - A refactored function returns an off-by-one result, but callers never see an exception — just subtly incorrect output.
    - A transformed database migration script accidentally drops a column used only in rare reporting queries, and those reports stop running without explicit failure notification.
  - **Model Bias, Drift, and Quality Variability:** Pretrained models can exhibit bias toward specific architectures or technologies, steering teams into suboptimal patterns. This risk increases in use cases involving heterogeneous technology stacks or nonstandard architectures. As underlying AI services evolve, teams must continually validate previous results to ensure consistent transformation quality.
  - **Security and Privacy Concerns:** Uploading proprietary or regulated source code to external AI services can expose sensitive IP or personal data. Generated code may inadvertently introduce new vulnerabilities (e.g., weak defaults, misconfigured permissions).
  - **Compliance and Licensing Pitfalls:** AI-generated snippets may be derived from open-source code with restrictive licenses, leading to inadvertent license violations. Automated tools may not enforce industry-specific compliance standards (HIPAA, PCI, GDPR) without custom policy integration. Modernization efforts targeting compliance-driven applications must prioritize these risks.
- **Process Risks**
  - **Integration Complexity and Operational Risk:** Embedding AI-powered transformations into existing CI/CD, governance, and toolchains can introduce new points of failure. Hosted AI platforms' service outages or rate limits can stall critical modernization workflows.
  - **Inadequate Testing and Validation Coverage:** Legacy systems often lack comprehensive tests, and developers must verify modernized code for functional equivalence. Sole reliance on AI-generated test cases can overlook legacy edge cases, undocumented logic, and integration dependencies (see [Manage Critical Risks of Using Generative AI to Augment Testing](#)).

## Alternatives

While AI-augmented modernization tools offer powerful automation, several alternatives suit different needs:

- **Manual code modernization** — Relies on in-house or consulting teams to manually refactor and rewrite legacy code by hand. This grants complete control over design patterns and coding standards and fosters deep knowledge transfer, but it's slow, labor-intensive, costly and prone to human error and scalability challenges.

A variation of the manual approach using scripting tools such as sed, Perl, or Python's re module handles minor, repetitive updates like import-statement changes efficiently and predictably. Developers can quickly customize patterns, but regex scripts falter on large or deeply structured tasks and require manual pattern definitions.

- **Code analysis and documentation tools** — Examine source code to discover and document its structure, complexity, health, and adherence to defined software architecture constructs and secure coding practices. Some specialized tools detect and report on particular aspects of the code, like its technical debt and structural complexity performance. For example, [CAST Imaging](#), [CodeScene](#) and [vFunction](#) provide architectural or graph diagrams that depict the system's functionality, structure, and runtime behavior. Code analysis and documentation tools have a limited focus and cover only part of the modernization workflow. AI-powered documentation tools are evolving, and the landscape is fragmented. See [How to Document Software With Generative AI](#).
- **Rule-based modernization tools** — Apply predefined transformation rules to enforce coding standards and translate old languages into modern ones. Solutions like OpenRewrite and Rocket Visual COBOL deliver deterministic, repeatable results, accelerating consistency and reducing errors. However, they demand a comprehensive ruleset and struggle with complex restructuring or nuanced scenarios, often retaining legacy structures.
- **AI code assistants** — Leverage large-language models in IDEs to suggest idiomatic API updates, refactor boilerplate, surface hidden dependencies, and draft documentation. Integrated in a human-in-the-loop workflow, these assistants accelerate small — to midscale refactoring and let developers focus on high-value design decisions. However, they don't replace expert architects or guarantee flawless output. See [Magic Quadrant for AI Code Assistants](#).

## Recommendations

Software engineering leaders responsible for modernization initiatives should:

- **Start with a clear modernization strategy** — Develop a coherent strategy that blends AI-assisted portfolio analysis with business capability mapping and target-state architecture. Prioritize initiatives by business impact, complexity, technical risk and strategic value.
- **Implement robust governance and compliance frameworks** — By defining clear policies for responsible AI use, data privacy, and ethics — particularly in regulated sectors such as banking, insurance, and healthcare. Specify permissible data inputs for external AI services, align with privacy regulations, and

ensure all AI-generated outputs meet organizational security and audit standards.

- **Set realistic expectations** — AI tools accelerate modernization tasks but won't magically rewrite your entire legacy estate. Treat AI as an assistant for code transformation and technical-debt identification, not a complete replacement of expert judgment.
- **Run pilot projects** — Begin with a representative, low-risk application to validate toolchain integration, measure productivity gains, and refine processes. Capture metrics on code transformation rates, defect reduction, and developer satisfaction.
- **Leverage AI for code understanding** — Use AI assistants to explain complex code, generate up-to-date documentation, and visualize module dependencies or "code health." These insights help architects and developers identify high-value refactoring targets and plan modernization roadmaps. Start with general-purpose AI code assistants to develop efficient software documentation practices, and consider specialist documentation generation tools if required.
- **Accelerate code transformation** — Use AI tools to transform code to target language or framework.
- **Enhance testing with AI** — Deploy AI tools to discover missing test cases, recommend existing ones, and auto-generate unit, integration, or regression tests. Comprehensive AI-driven test coverage is critical for safely refactoring and migrating legacy systems.
- **Automate ongoing code maintenance** — Use AI-augmented pipelines to Automate code-health monitoring, dependency updates, and technical debt alerts. Utilize AI for continuous improvement and to guide future modernization waves. See [How to Adopt a Continuous Application Modernization Strategy](#).
- **Monitor success indicators** — Track technical metrics (lead time for changes, test coverage percentage, defect density) and business KPIs (time-to-market, cost savings, customer satisfaction). Use dashboards and regular review cadences to align teams on progress and adjust course as needed. To establish and track the right metrics, see [Reframe Software Engineering Metrics for Agile and DevOps Success](#).

## Representative Providers

- [Amazon Q Developer Transform](#)
- [bloop.ai](#)
- [Cognition.ai](#)
- [CoreStory](#) (formerly Crowdbotics)
- [Devsu](#)
- [Diffblue](#)
- [dualboot 3PO](#)
- [Evolveware Intellisys](#)
- [GitHub Copilot app modernization](#)
- [Google Cloud Mainframe Modernization enhanced with Gemini](#)
- [IBM watsonx Code Assistant](#)
- [Konveyor AI \(Kai\)](#)
- [Mechanical Orchard Imogen](#)
- [Moderne](#)
- [phase change](#)
- [swimm.io](#)

⊕ Evidence

## Contributors

Arun Batchu, Matt Brasier, Howard Dodd, Tigran Egiazarov, Erin Khoo, Mukul Saha, Jim Scheibmeir, and Graham Wilkins.

© 2026 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's research organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner research may address legal and financial issues, Gartner does not provide legal or investment advice and its research should not be construed or used as such. Your access and use of this publication are governed by [Gartner's Usage Policy](#). Gartner prides itself on its reputation for independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "[Guiding Principles on Independence and Objectivity](#)." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

